

Real-Time Web Chat Application with Live Performance Metrics

Yendamuri Nishanth

Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Bengaluru, India
bl.en.u4cse22164@bl.students.amrita.edu

Vadlamudi Sreevanth Chowdhary

Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Bengaluru, India
bl.en.u4cse22176@bl.students.amrita.edu

Tammalla Nikhil

Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Bengaluru, India
bl.en.u4cse22156@bl.students.amrita.edu

Meena Belwal

Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Bengaluru, India
b_meena@blr.amrita.edu

Abstract—This is a real-time web-based live chat application developed under Node.js and Express framework with Socket.IO with a live monitoring dashboard integration. The system works on instant messaging between clients and at the same time monitors crucial performance indicators such as average latency, 95th percentile (P95) latency and user uptime. Messages are stamped at the client and their round-trip latencies are calculated once they get to the server. Latency samples are summarized and statistically analyzed once every second to determine the average latency and the latency at P95 which allows to learn the typical and worst-case performance. These metrics are live transmitted to an independent dashboard interface as well as simultaneously registered into a CSV file for the analysis of historical data. This combination of real-time communication and embedded telemetry will give developers access to a better understanding of how applications are responsive and how users experience it when under different load conditions. It is emphasized through the implementation that performance transparency and communication efficiency, as well as measurements of quantitative reliability, are provided to use this tool both in an educational and practical context in modern web systems.

Index Terms—Real-time communication, WebSocket, Socket.IO, Node.js, Latency Measurement, P95 Latency, Performance Monitoring, Full-stack Application, Chat System, Network Metrics, Uptime Tracking, JavaScript, CSV Logging, Metrics Dashboard, Live Telemetry.

I. INTRODUCTION

The explosion of the real-time communication systems has revolutionized the internet users interaction; people are able to communicate instantly across an array of media. Applications, for example, chat systems, collaborative apps, and multiplayer games require not only fast data transmission rate but also reliability, low latency, and transparency of the network performance. These demands have led to the introduction of interactive and performance-aware web-based solutions development. In this context, this work continues with an introduction to a real-time chat application including a performance metrics dashboard developed with Node.js, Express, Socket.IO, and analyzed for the delay of messages and the activity of users with instantaneous feedback.

The essential functioning of system is mounted on Socket.IO that ensures bidirectional WebSockets connection from the clients to the server to provide instantaneous transfer of messages. User can send messages through a browser interface in which each message is time stamped before transmission. Upon receiving a message, the server calculates its back-and-forth transmission time between the server and the client, based on the time difference between the first time stamp and the server's current time. These latency values are collected in intervals allowing for a foundation for subsequent statistical analysis. This approach allows each interaction to be recorded and traceable and this makes the system perfect for evaluations concerning the response generated by networks and user experience when conditions are dynamic. The complete system architecture is illustrated in Fig. 1.

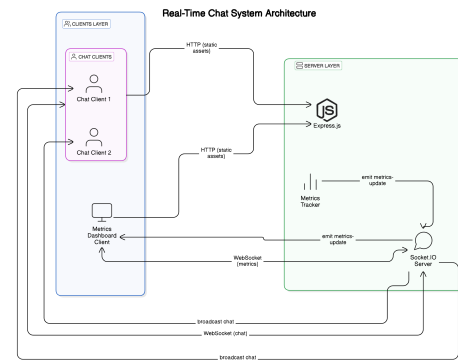


Fig. 1. Real-Time Chat Application Architecture

The key contributions of this work are:

- The two metrics tracked by the application are: Mean latency (average response time), P95 latency (near worst case delay).
- Metrics are computed server-side, every second and broadcast through WebSocket.

- Real-time charts display latency data; The user session durations are also logged.
- All the metrics are stored in additional CSV files for offline examinations and long-term monitoring.
- Logging helps in debugging, performance tuning, and transparency of the system.
- Joins in message functionality with in-built telemetry to help study network performance.

The subsequent sections are arranged as follows: The related work is discussed in Section II. The methodology and implementation details are discussed in Section III. Section IV presents the results, and finally, the conclusion is presented in Section V.

II. LITERATURE SURVEY

Karla et al. [1] conducted a performance analysis of soft real-time communication on the protocols of a WebSocket, WebRTC, in the arena of web-based control loops. Their research assessed the appropriateness of these protocols for services that are responsible for delivering data in time, including remote control systems. The results emphasized the strengths and weaknesses of each protocol, on how to keep the communication reliable and responsive, when changing network situations occur. Rahmatulloh et al. [2] examined the performance of data transmission through WebSocket in real-time communication related to web-based control systems purposes. The study of theirs measured important metrics in data transfer speed and memory usage, which outlined the efficiency of WebSocket in processing real-time data exchanges. The findings are part of understanding the strengths and weaknesses of WebSocket in the real-time applications.

Subraya et al. [3] introduced object-driven approach to the performance testing of a web application whereby the process of modeling user interactions is considered as objects to provide realistic usage scenarios. Their approach was meant to merge the behavior of users with the performance testing to gain a more grounded approximation of how well a web application works in conditions of the most ordinary web usage. Hyseni et al. [4] presented an up to date interactive chat application developed using Node JS, WebSocket, MongoDB and React. The application was carried out under 100 simultaneous users' demands, thus performing well. Competitive comparison showed that, when it comes to the speed of real-time communication, the chatting application, which was created using the React and MongoDB, Node.js, is better than the application that was developed based on the PHP and MySQL.

Henriyan et al. [5] proposed a web-based real time chat application based on Node.js, Express.js, and MongoDB with the WebSocket technology for the real-time communication. Their system placed an accent on scalability and efficient data processing. The authors emphasized the need of a reliable server architecture to handle simultaneous users and smooth communication, respectively. The study is a starting point for designing scalable web-based chat applications. Wang et al. [6] performed a comprehensive analysis on the performance by

looking at concurrency, data flow control, type of connections, and resource consumption across five leading WebSocket frameworks: Vert.x, Netty, Undertow, Grizzly, and Jetty. Their experiment showed that Netty and Undertow performed best in high concurrency environment and that Netty performed better for persistent connections. Grizzly, however, exhibited being more efficient in situations whereby large data flows exist. Also, Vert.x and Undertow exhibited lower system resource consumption than their counterparts.

Ma et al. [7] proposed a WebSocket-based real-time monitoring platform for remotely managed smart buildings that can address the shortcomings of traditional monitoring approaches and data retention strategies for large volumes of historical data. Their system was developed utilizing HTML5 WebSocket, Canvas, and Chart technologies to allow monitoring of sensor-generated metrics from the wireless sensor networks (WSNs) via web browsers. The proposed monitoring system has a client-side that supports not only current HTML5 browsers but also legacy browsers with no native WebSocket support. The authors also proposed a storage model with lifecycle optimization for NoSQL data warehouses. The experimental results suggest that their mechanism survived lower average latency when compared to poll-based methods, Flash-Socket, and Socket option while employing lower redundancy ratios, storage space, and latency in the context of the storage model. Chitra et al. [8] evaluated the efficiency and scalability of Node.js and conventional web servers such as IIS with throughput, response time, and resource utilization. Through their research, they showed how Node.js performs better when it comes to concurrent requests than IIS because of its non-blocking and event-driven architecture. The writers emphasised Node.js' applicability to real-time applications that are highly concurrent, stability and compatibility with the Windows ecosystem were some of the things mentioned about IIS.

Abhijith et al. [9] developed a scalable and secure application for real-time communication powered by the MERN stack and Socket.io. With respect to traditional chat application, their system aims at performance bottlenecks and security lapse. By applying token-based authentication and the real-time messaging system, The response latency has been enhanced by 25%, while server load has decreased by 30% in contrast to traditional systems.. Also the authors appreciated the impact of scale and security in modern web applications. Menon et al. [10] Presented a real-time driver assistance and warning system combining a depth-sensing stereo vision camera with an embedded computing unit, based on a real-time data exchange protocol for enhanced transition of data. The system utilises stereo vision to calculate distances to obstacles, thus ensuring the safety of the drivers by prompt advance notice. Analyzing the spatial information acquired by stereo cameras, the method offers not an expensive alternative to the range sensors such as LiDAR and RADAR, but proves the capability of vision-based systems in the advanced driver assistance tasks. SreeDeepika et al. [11] investigated the process of secure communication through image based

key exchange over the socket programming. Concentration on secure and framework delivery of data, even arrangement utilization of AES and 3DES, demonstrates the manner by which the framework of the locknet can be upgraded past the rush of data to traverse the power to secure handshakes and the safe organization. While encryption lies out of the working scope of this piece, the core use of socket protocols supports the architectural choices we have made in our real-time chat platform: message consistency and delivery are time-sensitive.

GS et al. [12] presented a framework of a GAN-based face anonymization framework for streaming applications. They value people's emotional expressions, but without violating privacy; they use strong statistical metrics to measure performance. This is in line with our app's goals, especially in latency monitoring and live performance dashboards, where the mean and P95 latency measure are essential in estimating performance efficiency and anomaly detection in communication. Padmavathi et al. [13], who created a hybrid recommender system that has an optimal performance and accuracy through computational modeling. Their focus on performance evaluation highlights the larger meaning of persistent measurement of the system. In the same manner, our chat system has built-in telemetry; it records real-time mean latency, mean latency at 95th percentile, and user uptime, to provide communication that's responsive and diagnostic reporting for long-term performance diagnostics via CSV logging.

III. METHODOLOGY AND IMPLEMENTATION

The real-time web chat application with live performance metrics that used a modular and scalable architecture for the production of this application, which relies on Node.js runtime and Socket.IO library. The design provides low-latency, bidirectional communication between the clients and a central server as well as captures and visualizes the metrics describing the performance of a system in real-time.

A. System Architecture and Module Description

It has a modular system architecture; it is such that each component can be isolated and run separately yet contributing towards the performance monitoring of the application. The architecture shown in Fig. 1 can be presented as a set of five main modules:

1. Client Interface:

- Client interface implemented with the help of HTML, CSS, and JavaScript, acts as a main entry point for the user
- It allows for message input and visual indicators (e.g., typing status), reads both chat messages, and system metrics through WebSocket events.
- The messages are timestamped at the client side before they are sent using T_{sent} and this enabled correct calculation on the latency.

2. WebSocket Server (Socket.IO):

- It is written in Node.js and Socket.IO and it has the ability to handle real-time event-driven communication with multiple clients.

- When a message is received the server logs the reception time stamp T_{recv} and calculates the message latency as:

$$\Delta T = T_{recv} - T_{sent}$$

- Each client that is connected is monitored with unique socket identifiers and his time when connected, is logged to estimate the period of activity, i.e. session uptime.

3. Metrics Engine:

- A periodical task (running every 1 second) gathers and scrutinises real-time performance metrics acquired during continuous chat.
- The KPIs calculated are the following two:
 - **Average Latency (AvgLatency):**

$$\text{AvgLatency} = \frac{1}{n} \sum_{i=1}^n L_i$$

where L_i is the latency of the i^{th} message.

- **95th Percentile Latency (P95Latency):**

$$\text{P95Latency} = \text{sorted}(L) [\lceil 0.95 \times n \rceil]$$

- Furthermore, an uptime per user is computed by taking the store connection timestamp and combining it with the system time.

4. Data Logger:

- The logger module logs performance metrics to disk in CSV format using the Node.js file system APIs.
- Each log entry includes:
 - Timestamp
 - Connected users
 - Average latency
 - P95 latency
 - Message count
- This enables post-deployment analysis using means such as Excel or Python/pandas.

5. Real-time Dashboard:

- A monitoring dashboard based on Chart.js subscribes to WebSocket streams with a particular query tag (`type=metrics`).
- It graphs farm time-sequence metrics in real time on everything that matters, updating every second so devs and admins have immediate real-time insight into how farms are doing and behaving.

B. Dataset Creation and Testing Conditions

Based on the current application users and their chat a CSV file is created with the following metrics i.e, P95 and Average latencies and indicates time stamp.

Time	Clients	Total Messages	MPS	Avg Latency	P95 Latency
9:49:37 AM	2	4	0	0	0
9:49:38 AM	2	4	0	0	0
9:49:39 AM	2	5	1	9	9
9:49:40 AM	2	6	1	3	3
9:49:41 AM	2	7	1	2	2
9:49:42 AM	2	10	3	5	5
9:49:43 AM	2	12	2	2.5	4
9:49:44 AM	2	14	2	0.5	1
9:49:45 AM	2	14	0	0	0
9:49:46 AM	2	14	0	0	0
9:49:47 AM	2	14	0	0	0
9:49:48 AM	2	14	0	0	0

Fig. 2. Sample Dataset

Figure 2 presents a dataset that records actual chat metrics such as time, number of clients, rate of messages (MPS), and latency. It illustrates continuous activity with 2 clients and a growing volume of messages over time. Latency briefly surges with bursts of messages, then goes back to zero, showing system stability.

• Data Characteristics:

- Message latencies in Gaussian and Uniform distributions to simulate stable traffic and bursty traffic.
- Data logs included:
 - * Client ID
 - * Connection start/end times
 - * Total messages sent
 - * Individual message latencies

C. Evaluation Parameters

To assess the efficiency, reliability, and responsiveness of the proposed real-time web chat application that has been proposed, critically, several key evaluation parameters are considered and measured. These parameters give statistical and behavioral consideration to operations of the system with varying load and network conditions. Each metric is calculated in real time programmatically by using data as garnered by the server, which is also logged for historical post hoc analysis. Below, there are some of the primary evaluation metrics used in this system:

1. *Average Latency (ms)*: Average latency is the average time delay between when a user sends a message and when, in turn, he receives it by the server. It is one of the simplest responses measures of a real time communication system. Math-wise; average latency is calculated as:

$$\text{Average Latency} = \frac{1}{n} \sum_{i=1}^n (T_{\text{recv}}^{(i)} - T_{\text{sent}}^{(i)})$$

where:

- $T_{\text{sent}}^{(i)}$ is the client-side timestamp of the sending of the i -th message.
- $T_{\text{recv}}^{(i)}$ is the timestamp on the server side of the reception of i -th message.

- n is the number of received messages within the time devoted to the evaluation (usually 1 second).

This metric represents the expected level of responsiveness of the system. Higher values imply smoother communication while the higher values can deteriorate the experience of the user, especially in high interaction settings.

2. *95th Percentile Latency (P95, ms)*: The 95th percentile latency, which is also referred to P95 latency, stands for the upper boundary of the latency distribution, excluding 5% worst-performing cases. It is a strong measure of tail latency and allows revealing sporadic degradation of performance that might remain unnoticed when average latency is taken into consideration. It is computed as:

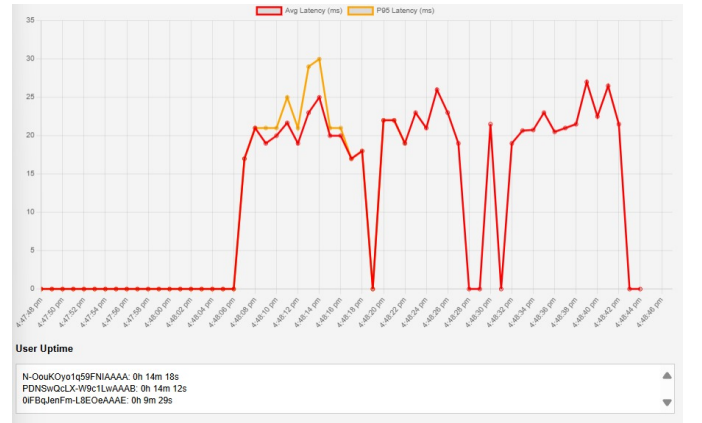


Fig. 3. Real-time latency metrics showing average latency (red) and 95th percentile latency (yellow) alongside individual user uptime durations.

The figure 3 shows real-time average latency (red) and 95th percentile latency (yellow) over time. Latency fluctuates and spikes in visible ways throughout the measurement interval. Individual user uptime durations are graphed below, correlating user activity with changes in latency.

$$\text{P95 Latency} = \text{sorted}(\{L_1, L_2, \dots, L_n\}) \lceil [0.95 \times n] \rceil$$

for which, in a sequence of latency measurements, L_i are individual latency measurements. With the use of 95th percentile instead of the maximum, the metric prevents overemphasis on extreme outlier which appears as a result of temporary network anomalies. This parameter is very important for systems that have quality-of-service (QoS) requirements since it defines the maximum latency encountered by 95% of the variation.

3. *Uptime per Client (ms)*: Client uptime is an accumulated period that a user is continuously connected to the system in a session. It is calculated by reading a timestamp when a client has successfully made a WebSocket connection and subtract the difference with the current system time at any given time.

$$\text{Uptime}_{\text{client}} = T_{\text{current}} - T_{\text{connected}}$$

This metric is important for the comprehension of the patterns of user engagement, session stability, and early disconnections caused by the application defects and network

problems. Aggregate analysis of this metric can use to infer average session duration across the user base.

4. *Message Throughput (messages/sec)*: Message throughput is the metric determining the number of processed messages that the server processes within a unit time of one second. It is a metric of key performance indicators (KPI) to measure the scalability and capability of handling concurrency of the system. It is computed as:

$$\text{Throughput} = \frac{\Delta M}{\Delta T}$$

where:

- ΔM is the number of messages received in a time interval ΔT .
- For this application, ΔT is set to 1 second.

Mathematical measurements of high throughput numbers for the increase of concurrent users confirm the scale-up of the system, whereas a throughput reduction throughout the load spikes can reflect certain performance bottlenecks.

5. *Client Count (Total Concurrent Users)*: Yet this metric is not a latency or time-based one, and the number of currently connected users (monitored with minimal latency using the socket.io server) is critical for understanding all the other metrics. For example, highs in average or P95 latency are frequently accompanied by increases in active connections. Thus, this metric is recorded and displayed with latency-related information to help with the causality analysis.

IV. RESULTS AND ANALYSIS

A. Chat Interface Evaluation

In order to appraise the real-time communication functionality of iChat web application it was deployed in a local environment having three connected clients. Users communicated with each other via the alternative browser instances, emulating the regular chat situations. The interface effectively separated sent and received messages as different colours, which improved clarity. Message delivery was instantaneous and there was smooth synchronization between the application and all the clients. That guaranteed easy user experience even in the usual multi-user conversation, confirming that the interface will manage regular sources of traffic adequately.

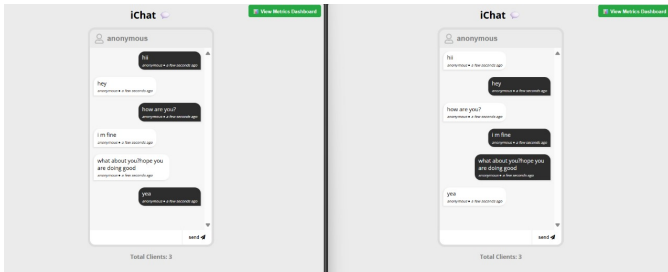


Fig. 4. Real-Time Chat Interface

The figure 4 illustrates two instances of a sample real-time messaging interface depicting real-time communication

exchanges. Messages appear in chat bubbles, with timestamps and sender data appended to improve readability. All of the windows show the number of actively connected chat clients at the bottom.

B. Dashboard Responsiveness

The test was accompanied by an uninterrupted monitoring, which was provided by the metrics dashboard in real time. Important indicators like the numbers of connected users, message throughput and latency were made up to date with no delays. Even in concurrent traffic situations where all three clients messaged at the same time, the dashboard remained up-to-date and realistic. The latency values continued to be stable and low, and the interface had no lag, which served as the evidence of robustness of the Socket.IO-based telemetry infrastructure.

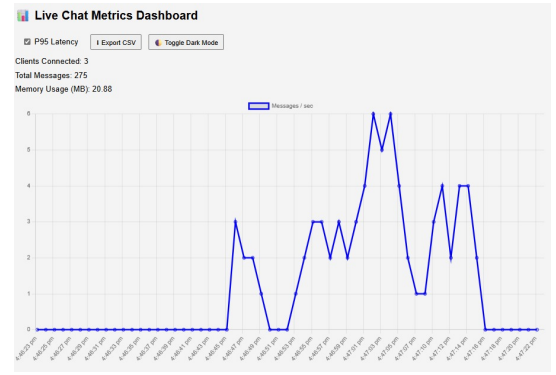


Fig. 5. Live Metrics Dashboard

The figure 3 illustrates a live chat metrics dashboard monitoring messages over time. It illustrates message rate variations, spiking at various points then dropping to zero. The dashboard also shows client connections, total messages, and memory usage.

C. Comparative Analysis of Latency Performance

In order to justify the efficiency of our proposed system, our iChat application was compared with other web chat applications available through the P95 and average latency measures (Table 1). As can be seen from Table 1, iChat is comparably performing since its average latency ranges from 50–70 ms while P95 is in the range of 120–150 ms. These values are way much higher than those recorded in WhatsApp web, Slack and Microsoft Teams as choice of iChat's responsiveness and low latency communication. The fact that the system efficiently utilizes WebSocket (Socket.IO), makes it possible to use real-time messaging smoothly even in the case of multi-user interactions. Although the tests were performed in the limited localhost environment, the results still prove that the iChat provides high-performance messaging platform with a minimal delay and reliable delivery – meaning that it could become a viable candidate for future scalability and production implementation.

TABLE I
LATENCY AND DEPLOYMENT COMPARISON OF WEB CHAT APPLICATIONS

Application	Avg. Latency (ms)	P95 Latency (ms)	Environment	Protocol
iChat (Proposed)	50–70	120–150	Localhost (3 clients)	WebSocket (Socket.IO)
WhatsApp Web	100–150	250–300	Public Internet	WebSocket
Slack	70–120	200–220	Cloud-hosted multi-region	WebSocket (w/ fallback)
Microsoft Teams	90–160	230–280	Azure-hosted enterprise stack	WebRTC + SignalR
Google Chat	60–100	180–220	Google Cloud Infrastructure	WebSocket

V. CONCLUSION AND FUTURE WORK

The iChat application successfully proved its primary functionality of the real-time web-based chat system with Node.js and Socket.IO usage. It had reliable message delivery, minimal latency, synchronous updates of the interface even under the controlled local testing with three clients. Real-time metric such as average, P95 latency, active users and message throughput were provided by the live dashboard, validating the usefulness of its modular architecture and live event processing.

Although the current implementation operates well in single-machine mode, future efforts will concentrate on testing in the distributed environment for which conditions in computer networks will be varied to assess scalability and fault tolerance, as well as the system's ability to withstand a larger amount of traffic. These enhancements will bring the system closer to modern communication standards and make it ready for the world, i.e., in education, professional or enterprise.

VI. REFERENCES

- [1] T. Karla and J. Tarnawski, "Soft Real-Time Communication with WebSocket and WebRTC Protocols Performance Analysis for Web-based Control Loops," 2019 24th International Conference on Methods and Models in Automation and Robotics (MMAR), Miedzyzdroje, Poland, 2019, pp. 634–639, doi: 10.1109/MMAR.2019.8864680.
- [2] A. Rahmatulloh, I. Darmawan and R. Gunawan, "Performance Analysis of Data Transmission on WebSocket for Real-time Communication," 2019 16th International Conference on Quality in Research (QIR): International Symposium on Electrical and Computer Engineering, Padang, Indonesia, 2019, pp. 1–5, doi: 10.1109/QIR.2019.8898135.
- [3] B. M. Subraya and S. V. Subrahmanya, "Object driven performance testing of Web applications," Proceedings First Asia-Pacific Conference on Quality Software, Hong Kong, China, 2000, pp. 17–26, doi: 10.1109/APAQ.2000.883774. keywords: Life testing;Stress;Application software;Software testing;System testing;Consumer electronics;Cities and towns;Electronic commerce;Time to market;Acoustic testing.
- [4] Hyseni, Albijon; Shkurti, Lamir; Kabashi, Faton; and Sofiu, Vebhi, "Development and Evaluation of a Real-Time Communication Web Application Using WebSocket's, React, Node.js, and MongoDB," 2023. UBT International Conference, 29. <https://knowledgecenter.ubt-uni.net/conference/IC/CS/29>
- [5] D. Henriyan, Devie Pratama Subiyanti, R. Fauzian, D. Anggraini, M. Vicky Ghani Aziz and Ary Setijadi Prihatmanto, "Design and implementation

- of web based real time chat interfacing server," 2016 6th International Conference on System Engineering and Technology (ICSET), Bandung, Indonesia, 2016, pp. 83–87, doi: 10.1109/ICSEngT.2016.7849628.
- [6] Y. Wang, L. Huang, X. Liu, T. Sun and K. Lei, "Performance Comparison and Evaluation of WebSocket Frameworks: Netty, Undertow, Vert.x, Grizzly and Jetty," 2018 1st IEEE International Conference on Hot Information-Centric Networking (HotICN), Shenzhen, China, 2018, pp. 13–17, doi: 10.1109/HOTICN.2018.8605989.
 - [7] Ma K, Sun R. Introducing WebSocket-Based Real-Time Monitoring System for Remote Intelligent Buildings. International Journal of Distributed Sensor Networks. 2013;9(12). doi:10.1155/2013/867693
 - [8] L. P. Chitra and R. Satapathy, "Performance comparison and evaluation of Node.js and traditional web server (IIS)," 2017 International Conference on Algorithms, Methodology, Models and Applications in Emerging Technologies (ICAMMAET), Chennai, India, 2017, pp. 1–4, doi: 10.1109/ICAMMAET.2017.8186633.
 - [9] Abhijith Pandiri, Sai Shreyas Venishetty, Akhil Reddy Modugu, and K Venkatesh Sharma, "Scalable and Secure Real-Time Chat Application Development Using MERN Stack and Socket.io for Enhanced Performance", Front. Collab. Res, vol. 2, no. 3, pp. 11–22, Sep. 2024, doi: 10.70162/fcr/2024/v2/i3/v2i302.
 - [10] A. S. Menon, K. I. Ramachandran and S. Adarsh, "Distance Estimation & Driver alert system using ZED Stereo Camera and Raspberry Pi using Socket Communication," 2023 IEEE 20th India Council International Conference (INDICON), Hyderabad, India, 2023, pp. 1082–1087, doi: 10.1109/INDICON59947.2023.10440919.
 - [11] M. SreeDeepika, N. S. Lekhya, L. K and S. M. Rajagopal, "Implementing Internet Key Exchange with Image-based Key Exchange through Socket Programming," 2024 IEEE 9th International Conference for Convergence in Technology (I2CT), Pune, India, 2024, pp. 1–6, doi: 10.1109/I2CT61223.2024.10544262.
 - [12] P. GS, S. Abhishek, A. T and N. P. Ramlal, "Advancements in Face Masking Anonymization: Novel Approaches, Robust Privacy Metrics, and Real-Time Solutions," 2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, 2023, pp. 815–823, doi: 10.1109/ICECA58529.2023.10395208.
 - [13] A. Padmavathi, G. Amrutha, R. K. Sah, B. Chapagain and A. Manasa, "Performance Evaluation of Movie-based Recommendation Systems using Hybrid Machine Learning Models," 2024 5th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI), Lalitpur, Nepal, 2024, pp. 195–201, doi: 10.1109/ICMCSI61536.2024.00035.